PROJECT REPORT

IBM Skills Build Internship in Collaboration with Naan Mudhalvan

Project Title

AI HR Recruitment Assistant: An Agentic AI System for Resume Screening, Candidate Matching and Interview Preparation

---

Name: T MAHA LAKSMI

Register Number: 923823104030

Department & Year: CSE & FINAL YEAR

Naan Mudhalvan ID (NM ID): C2B2030DE37C8C7A5D345C44DD1CFDBF

College Name: MANGAYARKARASI COLLEGE OF ENGINEERING

College Code: 9238

1. Project Overview

1.1 Project Title

AI HR Recruitment Assistant — An Agentic AI system for resume screening, candidate matching, and interview question generation.

1.2 Problem Statement

Manual resume screening is slow, inconsistent, and difficult to scale. Recruiters reviewing large candidate pools often apply subjective judgement that varies from one resume to the next, spend disproportionate time on unqualified applicants, and walk into interviews without a structured, evidence-based set of questions tailored to each candidate's specific strengths and gaps. There is a need for a tool that evaluates every candidate against the same job description consistently, surfaces exactly which required skills are present or missing, and prepares the interviewer accordingly.

1.3 Brief Description of the Project

The AI HR Recruitment Assistant is an agentic AI application that automates candidate screening. A recruiter pastes a job description and adds one or more candidate resumes; the system evaluates each candidate against the role, produces a ranked screening report with a fit score, matched and missing skills, and a recruiter summary, and can generate a tailored set of interview questions for any candidate on demand. Two implementations were built: a cloud-based version using the Anthropic Claude API with structured tool calling inside an interactive React interface, and a fully local, offline variant using Ollama for both embeddings and generation, demonstrating the same agentic architecture independent of the underlying model provider.

2. Objectives & Proposed Solution

2.1 Project Objectives

- Automatically evaluate a candidate's resume against a given job description and produce a consistent fit score.
- Identify which required skills a candidate matches and which are missing, rather than a single opaque number.
- Rank multiple candidates for the same role so recruiters can prioritise review.

- Generate interview questions tailored to each candidate's specific strengths and gaps.
- Ground every AI output in the candidate's actual resume text, avoiding fabricated or generic assessments.
- Demonstrate the same solution running on both a cloud AI model and a fully local, offline AI model.

2.2 How the Agentic AI Solution Works

The system is built as a single orchestrating agent — the HR Recruitment Screening Agent — that carries out a multi-step workflow rather than a single prompt-response exchange. For each candidate, the agent retrieves the most relevant parts of the resume in relation to the job description (Retrieval-Augmented Generation, or RAG), then calls a structured evaluation tool that returns a fit score, matched skills, missing skills, and a verdict. Once every candidate has been screened, the agent ranks them by score. On request, it repeats the retrieval step and calls a second tool to generate interview questions tailored to that candidate's specific gaps. Because outputs are forced through defined schemas (tool calling) rather than free text, results are reliable, structured, and directly renderable in the interface.

2.3 Key Features

- Job description and multi-candidate resume intake in a single working session.
- One-click screening of an entire candidate stack against the job description.
- Ranked screening report with fit score, verdict stamp, matched skills, and missing skills per candidate.
- On-demand, candidate-specific interview question generation covering technical, behavioral, gap-probing, and culture-fit angles.
- Two interchangeable AI backends: Anthropic Claude (cloud, tool-use) and Ollama (local, offline RAG).
- Distinct, case-file-styled interface designed for quick recruiter scanning.

3. Implementation & Results

3.1 Technologies / Tools Used

Item	Purpose
React	Interactive front-end interface for the cloud-based version

Anthropic Claude API (claude-sonnet-4-6)	Cloud LLM performing evaluation and question generation via forced tool calls
Ollama (llama3.1)	Local LLM performing the same evaluation and generation, offline
Ollama (nomic-embed-text)	Local embedding model powering RAG retrieval
Python (requests)	HTTP client connecting the local agent to the Ollama REST API
Structured tool-calling schemas	Force both LLM backends to return reliable, parseable JSON
In-memory Vector Store	Stores embedded resume chunks and performs cosine-similarity retrieval
Tailwind utility classes	Layout and spacing for the React interface

3.2 Working Process

- Step 1 — Intake: the recruiter pastes the job description once and adds each candidate's resume to a working stack.
- Step 2 — Retrieve: for every candidate, the system embeds resume text and retrieves the chunks most relevant to the job description (RAG).
- Step 3 — Evaluate: the retrieved context is passed to the LLM through a forced structured tool, returning a fit score, matched/missing skills, and verdict.
- Step 4 — Rank: all evaluations are sorted by fit score into a single screening report.
- Step 5 — Prepare: for a selected candidate, the agent retrieves context again and calls a second tool to generate five tailored interview questions.

3.3 Screenshots / Output

Sample structured output produced by the working pipeline for a Backend Developer role (Java, Spring Boot, MySQL; Docker/CI-CD preferred), screening two candidates:

```
[
  {
    "candidate_name": "Arjun",
    "match_score": 78,
    "matched_skills": ["Java", "Spring Boot", "MySQL", "REST APIs"],
    "missing_skills": ["Docker", "CI/CD"],
    "verdict": "Strong Match"
  },
  {
    "candidate_name": "Divya",
    "match_score": 22,
    "matched_skills": [],
    "missing_skills": ["Java", "Spring Boot", "MySQL", "REST APIs",
      "Docker", "CI/CD"],
    "verdict": "Not a Fit"
  }
]
```

For the top-ranked candidate, Arjun, the agent generated interview questions including a direct probe of his missing Docker skill: “You haven't listed Docker experience — have you deployed

an application in a containerized environment, even informally?" alongside technical, behavioral, and culture-fit questions.

3.4 Results Achieved

- Both candidates in the test set were evaluated consistently against identical criteria, with Arjun correctly ranked above Divya (78 vs 22) based on relevant backend skills.
- Missing-skill detection correctly flagged Docker and CI/CD for Arjun despite his otherwise strong match, information a single overall score would have hidden.
- Interview questions generated were specific to the candidate's resume and the job description rather than generic, including a targeted gap-probing question.
- The same agent logic and tool schemas produced consistent behaviour across both the cloud (Claude) and local (Ollama) backends, confirming the architecture is model-agnostic.

4. Conclusion & Future Scope

4.1 Project Conclusion

The AI HR Recruitment Assistant successfully demonstrates how agentic AI, structured tool calling, and retrieval-augmented generation can be combined to automate a genuine hiring workflow. By grounding every evaluation in the candidate's actual resume and forcing structured outputs, the system produces consistent, explainable screening results and interview preparation that would otherwise take a recruiter significant manual effort. Building both a cloud-based and a fully local implementation reinforced that the underlying design pattern — retrieve, evaluate, rank, generate — is portable across different AI model providers.

4.2 Challenges Faced

- Ensuring the LLM reliably returned valid, schema-conforming JSON, particularly with the local Ollama model, which lacks native tool-calling and required prompt-enforced JSON formatting instead.
- Balancing chunk size and overlap during resume embedding so that retrieved context stayed relevant without losing surrounding meaning.
- Keeping the model's scoring honest rather than generously optimistic, which required explicit system-level instruction.
- Designing an interface that could present a ranked, multi-candidate report clearly without overwhelming the recruiter.

4.3 Future Enhancements

- Support resume file uploads (PDF/DOCX) instead of pasted text, with automatic text extraction.
- Persist screening history across sessions using durable storage, so past candidate pools can be revisited.
- Add bulk export of the screening report and interview questions to PDF or Word for sharing with hiring panels.
- Introduce bias and fairness checks on the scoring process to guard against unintended discriminatory patterns.
- Extend the agent with a scheduling tool that can propose interview slots directly from a connected calendar.